

Bloc 2 TD Initiation PL/pgSQL

Contexte	2
Exercice 1	2
Exercice 2	3
Exercice 3	6
Exercice 4	7
Exercice 5	8
Exercice 6	9
Exercice 7	12
Conclusion	12

Contexte

Ce TD va nous introduire aux **PL/pgSQL** avec une série d'exercices pour nous entraîner et à le comprendre.

(Les scripts seront disponible dans

"bvl_S3_B2_TD_Initiation_PL_pgSQL_files.sql")

Exercice 1

Il fallait créer une fonction qui calcule la longueur de deux chaînes de caractères fournies en argument et qui retourne la longueur la plus longue :

The screenshot shows a SQL IDE interface. The top pane, titled 'Query', contains the following SQL code:

```
1 CREATE OR REPLACE FUNCTION calculer_longueur_max(chaine1 VARCHAR, chaine2 VARCHAR)
2 RETURNS INTEGER AS $$
3 DECLARE
4     long1 INTEGER;
5     long2 INTEGER;
6 BEGIN
7     long1 := LENGTH(chaine1);
8     long2 := LENGTH(chaine2);
9
10    IF long1 >= long2 THEN
11        RETURN long1;
12    ELSE
13        RETURN long2;
14    END IF;
15 END;
16 $$ LANGUAGE plpgsql;
17
18 SELECT calculer_longueur_max('Voiture', 'Castor');
```

The bottom pane, titled 'Data Output', shows the result of the query. It includes a toolbar with icons for various actions and a table of results. The table has two columns: the function name and its return type. The first row shows the result of the function call.

Data Output	
calculer_longueur_max	integer
1	7

Exercice 2

Ce script doit compter et retourner le nombre d'occurrences d'un caractère dans un intervalle d'une chaîne de caractères. L'intervalle est indiqué par une position de départ et de fin dans la chaîne.

Version qui utilise “**FOR**” :

```
-- Exo2 (for) --
CREATE OR REPLACE FUNCTION nb_occurrences1(caractere VARCHAR, chaine VARCHAR, debut INTEGER, fin INTEGER)
RETURNS INTEGER AS $$
DECLARE
    compteur INTEGER := 0;
    i INTEGER;
    lettre VARCHAR;
BEGIN
    IF debut < 1 OR fin > LENGTH(chaine) OR debut > fin THEN
        RAISE EXCEPTION 'Intervalle invalide';
    END IF;

    FOR i IN debut..fin LOOP
        lettre := SUBSTR(chaine, i, 1);
        IF lettre = caractere THEN
            compteur := compteur + 1;
        END IF;
    END LOOP;

    RETURN compteur;
END;
$$ LANGUAGE plpgsql;

SELECT nb_occurrences1('n', 'banane', 2, 6);
```

Output Messages Notifications

Showing rows: 1 to 1 Page No:

nb_occurrences1	
integer	
	2

Version “LOOP/EXIT” :

```
-- Exo2 (loop/exit) --
CREATE OR REPLACE FUNCTION nb_occurrences2(caractere VARCHAR, chaine VARCHAR, debut INTEGER, fin INTEGER)
RETURNS INTEGER AS $$
DECLARE
    compteur INTEGER := 0;
    i INTEGER;
    lettre VARCHAR;
BEGIN
    IF debut < 1 OR fin > LENGTH(chaine) OR debut > fin THEN
        RAISE EXCEPTION 'Intervalle invalide';
    END IF;

    i := debut;

    LOOP
        EXIT WHEN i > fin;

        lettre := SUBSTR(chaine, i, 1);

        IF lettre = caractere THEN
            compteur := compteur + 1;
        END IF;

        i := i + 1;
    END LOOP;

    RETURN compteur;
END;
$$ LANGUAGE plpgsql;

SELECT nb_occurrences2('n', 'banane', 2, 6);
```

	nb_occurrences2	
	integer	
1		2

Version “WHILE” :

```
-- Exo2 (while) --
CREATE OR REPLACE FUNCTION nb_occurrences3(caractere VARCHAR, chaine VARCHAR, debut INTEGER, fin INTEGER)
RETURNS INTEGER AS $$
DECLARE
    compteur INTEGER := 0;
    i INTEGER;
    lettre VARCHAR;
BEGIN
    IF debut < 1 OR fin > LENGTH(chaine) OR debut > fin THEN
        RAISE EXCEPTION 'Intervalle invalide';
    END IF;

    i := debut;
    WHILE i <= fin LOOP
        lettre := SUBSTR(chaine, i, 1);
        IF lettre = caractere THEN
            compteur := compteur + 1;
        END IF;
        i := i + 1;
    END LOOP;

    RETURN compteur;
END;
$$ LANGUAGE plpgsql;

SELECT nb_occurrences3('n', 'banane', 2, 6);
```

	nb_occurrences3 integer 	
1	2	

Exercice 3

Cette fonction doit renvoyer le nombre de jours dans un mois avec une date fournie en entrée.

```
-- Exo3 --
CREATE OR REPLACE FUNCTION getNbJoursParMois(date_param DATE)
RETURNS INTEGER AS $$
DECLARE
    mois INTEGER;
    annee INTEGER;
BEGIN
    mois := EXTRACT(MONTH FROM date_param);
    annee := EXTRACT(YEAR FROM date_param);
    IF mois IN (1, 3, 5, 7, 8, 10, 12) THEN
        RETURN 31;

    ELSIF mois IN (4, 6, 9, 11) THEN
        RETURN 30;

    ELSIF mois = 2 THEN
        IF (annee % 4 = 0 AND annee % 100 <> 0) OR (annee % 400 = 0) THEN
            RETURN 29;
        ELSE
            RETURN 28;
        END IF;
    END IF;

    RETURN 0;
END;
$$ LANGUAGE plpgsql;

SELECT getNbJoursParMois('2023-08-09');
```

	getnbjoursparmois integer	
1	31	

Exercice 4

Les dates par défaut dans **MySQL** sont écrites comme “**AAAA/MM/JJ**”, le but de la fonction est de renvoyer la date au format **String** en mode “**JJ/MM/AA**”.

```
-- Exo4 --
CREATE OR REPLACE FUNCTION dateSqlToDatefr(date_param DATE)
RETURNS VARCHAR AS $$
DECLARE
    jour INTEGER;
    mois INTEGER;
    annee INTEGER;
    dateFr VARCHAR;
BEGIN
    jour := EXTRACT(DAY FROM date_param);
    mois := EXTRACT(MONTH FROM date_param);
    annee := SUBSTRING(EXTRACT(YEAR FROM date_param)::TEXT FROM 2);

    dateFr := CONCAT(jour, '/', mois, '/', annee);

    RETURN dateFr;
END;
$$ LANGUAGE plpgsql;

SELECT dateSqlToDatefr('2023-08-09');
```

Output Messages Notifications

datesqltodatefr
character varying 🔒

9/8/23

Exercice 5

Le but de cette fonction est de renvoyer le nom du jour de la semaine à partir d'une date en entrée.

```
-- Exo5 --
CREATE OR REPLACE FUNCTION getNomJour(date_param DATE)
RETURNS VARCHAR AS $$
DECLARE
    jour_num INTEGER;
    nom_jour VARCHAR;
BEGIN
    jour_num := EXTRACT(ISODOW FROM date_param);
    CASE jour_num
        WHEN 1 THEN nom_jour := 'Lundi';
        WHEN 2 THEN nom_jour := 'Mardi';
        WHEN 3 THEN nom_jour := 'Mercredi';
        WHEN 4 THEN nom_jour := 'Jeudi';
        WHEN 5 THEN nom_jour := 'Vendredi';
        WHEN 6 THEN nom_jour := 'Samedi';
        WHEN 7 THEN nom_jour := 'Dimanche';
        ELSE nom_jour := 'Inconnu';
    END CASE;

    RETURN nom_jour;
END;
$$ LANGUAGE plpgsql;

SELECT getNomJour('2023-08-09');
```

Output Messages Notifications



getnomjour
character varying 

Mercredi

Exercice 6

Injection de “**hhb_direct_v1.sql**” dans ma **BDD** actuelle, j’ai supprimé le début du script (qui génère la **BDD**) étant donné que la **BDD** existe déjà.

Aperçu :

```
--
-- TOC entry 140 (class 1259 OID 16515)
-- Dependencies: 3
-- Name: affecter; Type: TABLE; Schema: public; Owner: postgres; Tablespace:
--

CREATE TABLE affecter (
    num_client integer NOT NULL,
    date pg_catalog.date NOT NULL,
    num_agence integer
);

ALTER TABLE public.affecter OWNER TO postgres;

--
-- TOC entry 144 (class 1259 OID 16554)
-- Dependencies: 3
-- Name: agence; Type: TABLE; Schema: public; Owner: postgres; Tablespace:
--

CREATE TABLE agence (
    num_agence integer NOT NULL,
    nom_agence character varying(50),
    adresse_agence character varying(192),
    tel_agence character(10)
);
```

Il est ensuite demandé de créer une fonction qui renvoie le nombre de clients débiteur avec comme contrainte d'utiliser la fonction ***date_part()*** de **PostgreSQL**. (Elle sera utilisé pour vérifier la date de fermeture du compte et la d'aujourd'hui)

```
-- Exo6.1 --
CREATE OR REPLACE FUNCTION getNbClientsDebiteurs()
RETURNS INTEGER AS $$
DECLARE
    nb_debiteurs INTEGER;
BEGIN
    SELECT COUNT(DISTINCT p.num_client)
    INTO nb_debiteurs
    FROM posseder p
    JOIN compte c ON p.id_type = c.id_type AND p.num_compte = c.num_compte
    WHERE c.solde < 0
        AND (c.date_fermer IS NULL
            OR date_part('year', c.date_fermer) >= date_part('year', CURRENT_DATE));

    RETURN nb_debiteurs;
END;
$$ LANGUAGE plpgsql;

SELECT getNbClientsDebiteurs();
```

Output Messages Notifications

getnbclientsdebiturs integer 1

Showing rows: 1

Il est ensuite demandé de créer une fonction qui renvoie le nombre d'habitant vivant dans "x" ville ("x" étant la ville en argument)

```
-- Exo6.2 --
CREATE OR REPLACE FUNCTION getNbClientsVille(ville_param VARCHAR)
RETURNS INTEGER AS $$
DECLARE
    nb_clients INTEGER;
BEGIN
    SELECT COUNT(*)
    INTO nb_clients
    FROM client
    WHERE adresse_client ILIKE '%' || ville_param || '%';

    RETURN nb_clients;
END;
$$ LANGUAGE plpgsql;

SELECT getNbClientsVille('Lannion');
```

Output Messages Notifications

getnbclientsville	integer
	2

Exercice 7

Cette fonction permet de rajouter un client sans que l'utilisateur n'ait à entrer l'identifiant, la fonction va automatiquement récupérer le dernier identifiant de la BDD (pour les clients) et l'incrémenter pour ensuite l'ajouter

```
-- Exo7 --
CREATE OR REPLACE FUNCTION insererClient(
  p_nom VARCHAR,
  p_prenom VARCHAR,
  p_adresse VARCHAR,
  p_identifiant VARCHAR,
  p_mdp VARCHAR
)
RETURNS VARCHAR AS $$
DECLARE
  v_dernier_num INTEGER;
  v_nouveau_num INTEGER;
BEGIN
  -- Prend le dernier numéro existant --
  SELECT MAX(num_client)
  INTO v_dernier_num
  FROM client;

  -- Ajoute 1 au numéro --
  v_nouveau_num := v_dernier_num + 1;

  -- Insert la nouvelle ligne avec les paramètres mis au dessus --
  INSERT INTO client (num_client, nom_client, prenom_client, adresse_client, identifiant_internet, mdp_internet)
  VALUES (v_nouveau_num, p_nom, p_prenom, p_adresse, p_identifiant, p_mdp);

  -- Annonce le succès --
  RETURN 'Succès: Tuple enregistré avec le numéro ' || v_nouveau_num;
EXCEPTION
  -- Crash handle --
  WHEN OTHERS THEN
    RETURN 'Echecs : Le tuple n'a pas été enregistré.';
END;
$$ LANGUAGE plpgsql;
```

Vérification :

```
SELECT insererClient('ARMANT', 'Paul', '12 Rue des Lilas, 75000 PARIS', 'Parmant', 'azerty');
SELECT * FROM client;
```

num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy
6	ARMANT	Paul	12 Rue des Lilas, 75000 PARIS	Parmant	azerty

Conclusion

Ce TD nous a introduit au **PL/pgSQL** et a **pgAdmin 4** ainsi qu'à la création de fonction à l'intérieur d'une BDD.